

MODUL III

Form Request dan Refactor Service

Tujuan

- ◆ Mahasiswa mampu membuat FormRequest untuk validasi store & update Item/Category
- ◆ Mahasiswa mampu menulis custom validation messages
- ◆ Mahasiswa mampu menangani exception dan mengembalikan response JSON standar
- ◆ Mahasiswa mampu memindahkan CRUD logic ke Service class
- ◆ Mahasiswa mampu memanggil Service dari Controller

Materi

1. Validasi Input
 - 1.1. Laravel FormRequest (app/Http/Requests)
 - Buat dengan php artisan make:request StoreItemRequest
 - Method authorize(): return true/false (untuk otorisasi)
 - Method rules(): array aturan validasi (required, max, numeric, exists)
 - Method messages(): array pesan kustom
 - 1.2. Manfaat: pemisahan validasi agar controller bersih
2. Error Handling & Standard Response
 - 2.1. Try-catch di Controller untuk tangkap Exception
 - 2.2. app/Exceptions/Handler.php
 - Method render(): override untuk format JSON konsisten
 - 2.3. Standar response JSON:

```
{
  "status": "success"|"error",
  "data": ...,
  "message": "..."
}
```
3. Service Layer & Refactor
 - 3.1. Ide: pindahkan "business logic" (CRUD) ke class Service (app/Services)
 - 3.2. Controller hanya bertugas
 - Menerima Request
 - Memanggil Service
 - Mengembalikan Response
 - 3.3. Manfaat: kode terstruktur, mudah diuji, terpisah fokus setiap file.

Praktikum

1. Pastikan Anda switch dari branch main ke branch

feature/validation-service:

```
cd C:\laragon\www\inventory
git checkout main
git pull origin main
git checkout -b feature/validation-service
```

2. Buat FormRequest untuk Item

2.1. StoreItemRequest

```
php artisan make:request StoreItemRequest
```

File: app/Http/Requests/StoreItemRequest.php

```
<?php
...
class StoreItemRequest extends FormRequest {
    public function authorize() {
        return true;
    }
    public function rules() {
        return [
            'name'          => 'required|string|max:255',
            'quantity'      => 'required|integer|min:0',
            'price'         => 'required|numeric|min:0',
            'category_id' => 'required|exists:categories,id',
        ];
    }
    public function messages() {
        return [
            'name.required'      => 'Nama item wajib diisi.',
            'quantity.integer'   => 'Jumlah harus angka bulat.',
            'price.numeric'      => 'Harga harus berupa angka.',
            'category_id.exists' => 'Kategori tidak ditemukan.',
        ];
    }
}
```

2.2. UpdateItemRequest

```
php artisan make:request UpdateItemRequest
```

File: app/Http/Requests/UpdateItemRequest.php

```
<?php
...
class UpdateItemRequest extends FormRequest {
    public function authorize() {
        return true;
    }
    public function rules() {
        return [
            'name'          => 'sometimes|required|string|max:255',
            'quantity'      => 'sometimes|required|integer|min:0',
            'price'         => 'sometimes|required|numeric|min:0',
        ];
    }
}
```

```

        'category_id' =>
'sometimes|required|exists:categories,id',
    ];
}
public function messages() {
    return [
        'sometimes.required' => 'Field ini diperlukan saat
diubah.',
    ];
}
}

```

3. Buat FormRequest untuk Category

3.1. StoreCategoryRequest

php artisan make:request StoreCategoryRequest

File: app/Http/Requests/StoreCategoryRequest.php

```

<?php
...
class StoreCategoryRequest extends FormRequest {
    public function authorize() {
        return true;
    }
    public function rules() {
        return [
            'name' => 'required|string|unique:categories,name'
        ];
    }
    public function messages() {
        return [
            'name.unique' => 'Nama kategori sudah ada.'
        ];
    }
}

```

3.2. UpdateCategoryRequest

php artisan make:request UpdateCategoryRequest

File: app/Http/Requests/UpdateCategoryRequest.php

```

<?php
...
class UpdateCategoryRequest extends FormRequest {
    public function authorize() {
        return true;
    }
    public function rules() {
        $id = $this->route('category');
        return [
            'name' =>
"required|string|unique:categories,name,{ $id}"
        ];
    }
}

```

4. Buat Service Classes

4.1. ItemService

Buat file: app/Services/ItemService.php

```
<?php
...
class ItemService {
    public function all(): Collection {
        return Item::with('category')->get();
    }
    public function find(int $id): Item {
        return Item::with('category')->findOrFail($id);
    }
    public function create(array $data): Item {
        return Item::create($data);
    }
    public function update(int $id, array $data): Item {
        $item = Item::findOrFail($id);
        $item->update($data);
        return $item;
    }
    public function delete(int $id): void {
        Item::destroy($id);
    }
}
```

4.2. CategoryService

Buat file: app/Services/CategoryService.php

```
<?php
...
class CategoryService {
    public function all(): Collection {
        return Category::all();
    }
    public function find(int $id): Category {
        return Category::findOrFail($id);
    }
    public function create(array $data): Category {
        return Category::create($data);
    }
    public function update(int $id, array $data): Category {
        $cat = Category::findOrFail($id);
        $cat->update($data);
        return $cat;
    }
    public function delete(int $id): void {
        Category::destroy($id);
    }
}
```

5. Refactor Controllers untuk pakai Request & Service

5.1. ItemController

File: app/Http/Controllers/ItemController.php

```
<?php
```

```
...
```

```
class ItemController extends Controller {
    protected ItemService $svc;
    public function __construct(ItemService $svc) {
        $this->svc = $svc;
    }
    public function index() {
        return response()->json([
            'status' => 'success',
            'data'    => $this->svc->all(),
            'message' => 'Berhasil menarik semua data Item'
        ]);
    }
    public function store(StoreItemRequest $req) {
        $item = $this->svc->create($req->validated());
        return response()->json([
            'status' => 'success',
            'data'    => $item,
            'message' => 'Item berhasil dibuat'
        ], 201);
    }
    public function show($id) {
        try {
            $item = $this->svc->find($id);
            return response()->json([
                'status' => 'success',
                'data'    => $item,
                'message' => 'Berhasil menarik satu data Item'
            ]);
        } catch (\Exception $e) {
            return response()->json([
                'status' => 'error',
                'data'    => null,
                'message' => $e->getMessage()
            ], 404);
        }
    }
    public function update(UpdateItemRequest $req, $id) {
        $item = $this->svc->update($id, $req->validated());
        return response()->json([
            'status' => 'success',
            'data'    => $item,
            'message' => 'Item berhasil diperbarui'
        ]);
    }
    public function destroy($id) {
        $this->svc->delete($id);
        return response()->json([
            'status' => 'success',
```

```

        'data'      => null,
        'message' => 'Item berhasil dihapus'
    ], 204);
}
}

```

5.2. CategoryController

File: app/Http/Controllers/CategoryController.php

```

<?php
...
class CategoryController extends Controller {
    protected CategoryService $svc;
    public function __construct(CategoryService $svc) {
        $this->svc = $svc;
    }
    public function index() {
        return response()->json([
            'status' => 'success',
            'data'   => $this->svc->all(),
            'message' => 'Berhasil menarik semua data Kategori'
        ]);
    }
    public function store(StoreCategoryRequest $req) {
        $cat = $this->svc->create($req->validated());
        return response()->json([
            'status' => 'success',
            'data'   => $cat,
            'message' => 'Kategori berhasil dibuat'
        ], 201);
    }
    public function show($id) {
        try {
            $cat = $this->svc->find($id);
            return response()->json([
                'status' => 'success',
                'data'   => $cat,
                'message' => 'Berhasil menarik satu data kategori'
            ]);
        } catch (\Exception $e) {
            return response()->json([
                'status'=>'error',
                'data'=>null,
                'message'=>$e->getMessage()
            ], 404);
        }
    }
    public function update(UpdateCategoryRequest $req, $id) {
        $cat = $this->svc->update($id, $req->validated());
        return response()->json([
            'status' => 'success',
            'data'   => $cat,
            'message' => 'Kategori berhasil diperbarui'
        ]);
    }
}

```

```

public function destroy($id) {
    $this->svc->delete($id);
    return response()->json([
        'status' => 'success',
        'data' => null,
        'message' => 'Kategori berhasil dihapus'
    ],204);
}
}

```

6. (Optional) Tangani exception global

File: app/Exceptions/Handler.php → method render()

```

public function render($request, Throwable $e) {
    if ($request->is('api/*')) {
        return response()->json([
            'status' => 'error',
            'data' => null,
            'message' => $e->getMessage(),
        ], method_exists($e, 'getStatusCode') ?
            $e->getStatusCode() :
            500);
    }
    return parent::render($request, $e);
}

```

7. Uji dengan Postman

Coba validasi gagal (kirim quantity=-1, lihat pesan)

The screenshot shows a Postman interface for a POST request to `http://127.0.0.1:8000/api/items`. The request body is set to `x-www-form-urlencoded` with the following data:

Key	Value	Description
name	Laptop	
quantity	-10	
price	1500	
category_id	1	

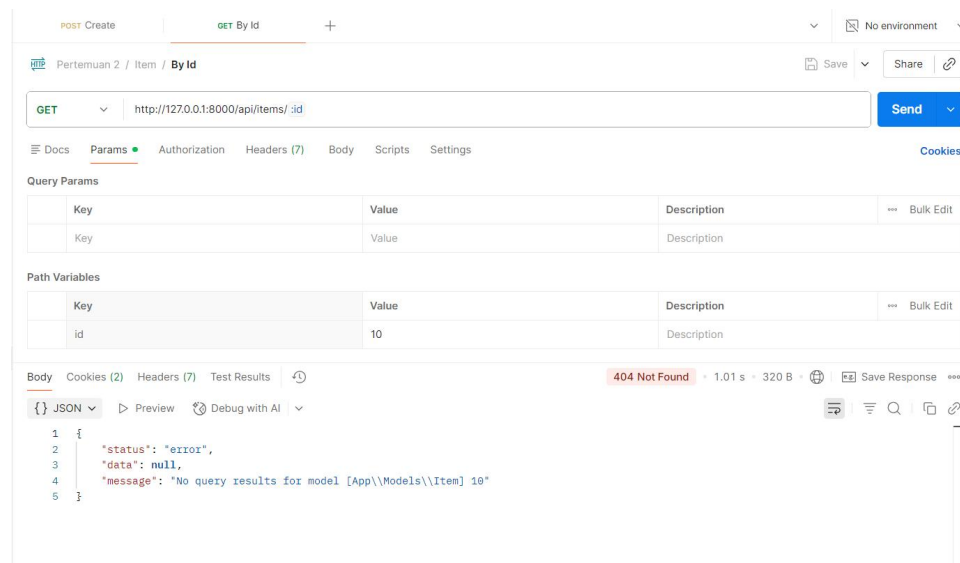
The response is a `500 Internal Server Error` with a status of 500, a response time of 796 ms, and a body size of 321 B. The response body is shown in JSON format:

```

{
  "status": "error",
  "data": null,
  "message": "The quantity field must be at least 0."
}

```

Coba GET/ items/{nonexistent} → dapat status 404 + JSON error



8. Commit & Push

```
git add .
git commit -m "Implement validation, error handling & service layer"
git push -u origin feature/validation-service
```

9. Pull Request

9.1. Buka Repository di GitHub

- Pergi ke <https://github.com/USERNAME/inventory>
- Kamu akan melihat notifikasi kuning: "**feature/validation-service had recent pushes**" → klik "**Compare & pull request**"

9.2. Isi Form Pull Request

- Base branch: main (branch tujuan merge)
- Compare branch: feature/validation-service (branch kamu)
- Title: Isi judul PR
- Setelah form diisi, klik tombol "Create pull request"

9.3. Review Kode (Melihat Perubahan)

- Klik tab "Files changed"
- Lihat perubahan kode:
 - Warna hijau = baris kode yang ditambah
 - Warna merah = baris kode yang dihapus

9.4. Merge Pull Request

- Klik tombol "Merge pull request"
- Klik "Confirm merge"